

Claude Code Prompt: Laundry Marketplace Platform (LaundryHub)

Copy everything below the line into Claude Code. It contains the full spec inline so Claude Code doesn't need to ask you anything — it should read this once and build end-to-end.

You are building a production-quality MVP of **LaundryHub**, a two-sided marketplace connecting **Clients** (customers) with independent **LaundryMart** vendors for on-demand wash, fold, and delivery service. Work fully autonomously: make every implementation decision yourself using the spec below, do not stop to ask clarifying questions, do not leave TODOs/placeholders in code, and self-review/fix build errors before finishing. There is no admin role/panel in this MVP.

Tech stack (do not deviate)

- **Frontend:** React.js (JS, ES6+), React Router, Context API for state (no Redux), Tailwind CSS with shared design tokens in `tailwind.config.js`, Axios with a centralized instance (JWT attach + 401 refresh interceptor).
- **Backend:** Node.js + Express, layered architecture — routes → controllers → services → Mongoose models.
- **Database:** MongoDB + Mongoose, with a `2dsphere` geospatial index on vendor location for geo-search.
- **Auth:** JWT access (15 min, held in memory client-side, never localStorage) + refresh (30 days, httpOnly+Secure+SameSite=Lax cookie, rotation with reuse detection), bcrypt.js (12 salt rounds) for password hashing.
- **File uploads:** Multer (memory storage, 5MB limit, jpg/png/webp only) streamed to Cloudinary — use a placeholder/mock Cloudinary service if no real API key is available, but keep the same interface so it's a one-line swap later.
- **Validation:** express-validator chains on every mutating route; reject unknown fields; strip `$ / .` - prefixed keys from bodies.
- **Payments:** Stripe + Stripe Connect (destination charges). Use Stripe test-mode keys/mock service if no live key is provided — the payment flow (create-intent → confirm → webhook → order

status flip) must still be fully wired and demoable with Stripe test cards, not skipped.

Design direction — do not build a generic marketplace template

Explicitly avoid: default Tailwind blue-500/indigo-600 buttons, generic rounded-xl card grids with drop shadows, stock "delivery van" or "laundry basket" clipart, centered hero with a big search bar that looks like every Yelp/Uber clone, System-UI/Inter-only typography with no personality.

Instead, build around this concept: "**fresh, tactile, and fast.**" Laundry is a sensory, physical service — clean linen, folded fabric, crisp light — so the visual language should feel warm, soft-textured, and efficient, like a well-designed local service brand, not a generic SaaS dashboard.

Concrete direction:

- **Typography:** pair a warm, rounded/humanist sans for headlines (e.g. Fraunces for display moments or a soft grotesk like General Sans / Cabinet Grotesk) with a clean, highly legible workhorse sans for UI and body text (e.g. Inter is fine for *body* copy only, but avoid using it for headlines — that's the generic tell). Give headline type real weight/size contrast.

- **Color palette:** soft off-white/linen background (not stark white), one deep grounding color (e.g. a deep teal, indigo, or forest — pick one and use it as the primary brand color across both roles), and a warm accent (e.g. a soft coral or marigold) reserved for CTAs, status badges, and highlights. Client and Vendor dashboards should share the same design system but can use subtle accent shifts to feel distinct (e.g. client = primary color forward, vendor = accent forward for "your business" framing).
- **Motifs:** soft-fold/wave shapes (evoking folded fabric) instead of generic blobs, pill-shaped status badges with clear color coding per order status, a real distinct empty-state illustration style (simple line-art, not stock icons), map pins and vendor cards with a soft-elevation (not harsh box-shadow) style.
- **Layout:** avoid perfectly centered symmetric hero sections everywhere — vary rhythm between sections (asymmetric feature sections, offset image/text splits). Dashboards should feel calm and information-dense without being cluttered: clear typographic hierarchy, generous but not wasteful whitespace, sticky sub-navigation within dashboards.
- **Motion:** subtle micro-interactions only — status timeline steps animate in on update, skeleton

loaders pulse gently, toasts slide in. No heavy scroll-jacking animation.

- Strictly one light theme — no dark mode needed for MVP.

Scope for this build

Implement the full **MVP scope** below. This is a large system — build it as a complete, coherent full-stack app with working end-to-end flows, not a static mockup. Where the spec doesn't dictate exact copy or micro-content, write real, specific copy yourself (no lorem ipsum).

Roles

Exactly two: **Client** and **LaundryMart** (vendor). No admin role/panel.

Backend folder structure

```
server/  
├─ config/           # db connection,  
cloudinary, stripe, env loader  
├─ controllers/      # auth, client,  
vendor, order, payment, review,  
notification  
├─ models/           # User, Vendor,  
Service, Order, Payment, Review,  
Notification  
├─ routes/           # one router per
```

```
resource, mounted under /api
├─ middleware/      # auth (JWT verify +
RBAC), validate, rateLimiter, errorHandler,
upload
├─ services/        # geocode.service,
stripe.service, cloudinary.service,
notification.service
├─ validations/     # express-validator
chains per route
├─ utils/           # asyncHandler,
ApiError, ApiResponse, logger
└─ server.js
```

Frontend folder structure

```
client/src/
├─ components/      # shared UI (Button,
Input, Modal, Toast, VendorCard...)
├─ pages/           # one folder per
route, e.g. pages/client/OrderTracking
├─ layouts/         #
ClientDashboardLayout,
VendorDashboardLayout, PublicLayout
├─ hooks/           # useAuth,
useGeolocation, useDebounce, useOrderStatus
├─ context/         # AuthContext,
NotificationContext, CartContext
├─ services/        # api/ axios
wrappers per resource (authApi, orderApi...)
├─ routes/          # AppRouter.jsx,
ProtectedRoute.jsx, RoleRoute.jsx
└─ utils/           # formatCurrency,
```

```
validators, dateHelpers
└─ assets/
```

Database schema (build exactly this shape)

- **Users (Clients):** name, email (unique, indexed), phone (E.164), passwordHash (select:false), role ('client'), addresses[] (max 10, embedded), stripeCustomerId, avatarUrl, isVerified, timestamps.
- **Vendors:** ownerName/email/phone/passwordHash, businessName, description (max 1000), location (GeoJSON Point, 2dsphere indexed), address, logoUrl/galleryUrls (max 10), hours[7], blackoutDates[], commissionRate (default 0.15), stripeAccountId, stripeOnboardingComplete, status ('pending_verification'/'active'/'deactivated'), ratingAvg/ratingCount (denormalized).
- **Services:** vendorId, name, category (wash_fold/dry_clean/bedding/ironing/other), unit (per_lb/per_item), price (cents), estTurnaroundHrs, isActive.
- **Orders:** clientId, vendorId, items[] (snapshot: serviceId, name, unit, unitPrice, qty, lineTotal), pickupWindow/deliveryWindow, address (snapshot), subtotal/platformFee/tip/tax/total (all server-computed, cents), status (enum below), statusHistory[], paymentId.

- **Payments:** orderId (unique), stripePaymentIntentId (unique), amount, applicationFee, status, refundedAmount.
- **Reviews:** orderId (unique – 1/order), clientId, vendorId, rating (1–5), comment (max 500), editableUntil (createdAt + 48h).
- **Notifications:** recipientId/recipientRole (polymorphic), type (order_update/new_order/new_review/payout/message), title/body/linkTo, isRead.

Order lifecycle (implement exact state machine)

pending_payment → confirmed → accepted → picked_up → in_progress → ready_for_delivery → delivered → completed, with terminal branches cancelled and disputed. Enforce legal-transition-only updates (reject illegal transitions with 409).

Business rules:

- Vendor has **15 minutes** to accept a new order or it auto-cancels with a full client refund.
- Client can cancel free until vendor marks picked_up; after that, cancellation requires vendor-approved partial refund.
- Reviews only allowed on completed orders, one per order, editable for 48 hours.

- Platform fee: 15% default commission, computed server-side, never trusted from the client.
- Geo-search default radius 8 miles, adjustable up to 25 miles.

Auth flows (both roles – implement identically per role-scoped routes)

- Register (with validation: email format+uniqueness async check, password ≥ 8 chars w/ 1 number+1 letter, phone via libphonenumber pattern), Login (rate-limited 5/15min per IP+email, 429 on 6th attempt with retry-after), Forgot Password (always returns 200, never reveals if email exists; signed single-use JWT reset token, 1hr expiry), Reset Password (verify signature+expiry+single-use, invalidate all refresh tokens), Logout (clear + blacklist refresh token).
- Vendor registration is a 3-step wizard: (1) owner info, (2) business info + geocoded address, (3) Stripe Connect Express onboarding redirect. Vendor status is `pending_verification` until the onboarding webhook flips it to `active` – pending vendors are excluded from client search.

Required pages – Public

Landing (hero w/ location-aware search, how-it-works, featured vendors, testimonials, CTA), About, Services

(static, CTA to search), Contact (form w/ inline states), FAQs (accordion by topic), Privacy/Terms.

Required pages — Client Dashboard

Overview, Nearby LaundryMarts (map+list toggle w/ geolocation fallback to ZIP entry), Search Vendors (filters + sort, URL-persisted query params), Vendor Details (closed-now banner logic), Book Laundry Service (item picker + running subtotal), Schedule Pickup (availability-constrained date/time picker, min-turnaround enforcement), Checkout (live recalculation), Payment (Stripe Elements, saved cards), Order Tracking (status timeline, polling/subscription for updates), Order History (paginated, reorder/review actions), Notifications, Profile (autosave on blur), Settings (delete-account requires typed confirmation + password re-entry).

Required pages — LaundryMart Dashboard

Overview (onboarding checklist until profile+service+Stripe complete), Manage Orders (kanban/table, 15-min response-time indicator), Manage Services (CRUD), Update Prices (bulk edit w/ % adjust + audit log), Manage Availability (weekly hours + blackout dates + capacity), Earnings (chart + payout history + CSV export), Customer Requests (message threads), Notifications, Business Profile,

Settings (Stripe Express deep-link, deactivate business).

API surface

Implement all endpoints exactly as specified, with the exact status-code behaviors:

- POST /api/auth/client/register|login, /api/auth/vendor/register|login, POST /api/auth/refresh, POST /api/auth/forgot-password, POST /api/auth/reset-password, POST /api/auth/logout
- GET /api/vendors?lat&lng&radius&sort&category&minRating (geo \$geoNear aggregation), GET /api/vendors/:id, PATCH /api/vendors/me, POST /api/vendors/me/media, PATCH /api/vendors/me/availability
- POST /api/services, PATCH /api/services/:id, DELETE /api/services/:id (owner-only, 403 otherwise)
- POST /api/orders, POST /api/payments/create-intent, POST /api/webhooks/stripe (signature-verified, source of truth for payment status – never trust client confirmation alone), GET /api/orders/me, PATCH /api/orders/:id/status, POST /api/orders/:id/cancel

- `POST /api/reviews`, `GET /api/notifications`,
`PATCH /api/notifications/:id/read`

Security requirements (implement all)

RBAC middleware reading role from verified JWT + resource-ownership checks; NoSQL injection sanitization middleware; React default escaping + CSP header (no `dangerouslySetInnerHTML` on user text); `SameSite=Lax` cookie + custom header for CSRF mitigation; CORS allowlist; all secrets via env vars with a `.env.example` file documenting every required key.

Component library

Build a real shared component library (not one-off styled divs per page): Navbar (role-aware), Hero, Search Bar (location autocomplete + "use my location"), Vendor Card, Service Card, Booking Form, Button (primary/secondary/destructive/ghost + loading state), Input/Select/Dropdown (shared error-message slot), Date & Time Picker (availability-aware), Payment Modal, Order Status Timeline (horizontal desktop / vertical mobile), Table (sortable, sticky header), Modal (focus-trapped, ESC to close), Toast, Skeleton Loader, Empty/Error States, Pagination, Footer.

Execution requirements

- Get both frontend and backend running successfully (`npm run dev` or equivalent) with a working MongoDB connection (use a local/in-memory Mongo or clearly documented connection string) before finishing.
- Seed the database with realistic sample data: at least 6 vendors (varied categories/locations/ratings), services per vendor, a couple of demo client accounts, and a few orders across different lifecycle states so the app is immediately explorable end-to-end without manual setup.
- Responsive at 320px–2560px; dashboards collapse to a mobile-friendly nav below 1024px.
- Accessibility: keyboard-navigable, visible focus states, form errors announced via `aria-live`.
- Write a `.env.example` for both client and server covering every required key (Mongo URI, JWT secret, Stripe keys, Cloudinary keys, Google Geocoding key, etc.), and a top-level `README.md` explaining how to run everything, what's mocked vs. live-service-ready, and the demo account credentials.
- When finished, give a concise summary of what was built, any services that are mocked pending

real API keys, and exact run instructions — do not
ask for approval mid-build.